CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 14: Context Fork

> **Exam objective:** Apply the context: fork frontmatter option to Skill and slash command configurations that should execute in an isolated subagent context, preventing cross-contamination of session state.

Q1.

A slash command performs a deep repository audit that often loads hundreds of files and lengthy intermediate reasoning. The main session should stay focused on the original task afterward. Which frontmatter choice is best?

A. Omit any context control so all audit state remains in the main session B. Use `context: fork` so the audit runs in an isolated subagent context C. Enable broader tool access instead D. Add more examples to the prompt but keep the same shared context

Answer: B **Explanation:** `context: fork` is designed for exactly this case: isolating a heavy or exploratory task so its state does not contaminate the parent session. Broader permissions or extra examples do not address context pollution. Leaving everything in the main session can degrade later interactions.

Q2.

A reusable Skill gathers debugging evidence, but the project owner does not want those exploratory steps and transient assumptions to influence subsequent architecture planning in the main thread. What should they do?

A. Put the Skill in the same shared context for continuity B. Apply `context: fork` to the Skill configuration C. Remove all tool calls from the Skill D. Convert the Skill into a plain text snippet

Answer: B **Explanation:** Forking the context isolates the debugging workflow and its intermediate state from the main thread. That preserves continuity where it matters while avoiding bleed-over from speculative investigation. Removing tools or simplifying the Skill does not solve the contamination problem directly.

Q3.

Which scenario most strongly suggests using `context: fork`?

A. A tiny command that reformats a single sentence and should build on current conversation context B. A task that needs a self-contained subagent to explore an alternative approach without affecting the parent session C. A command whose only purpose is to repeat the last assistant response D. A prompt that depends on inheriting every transient assumption from the main thread

Answer: B **Explanation:** `context: fork` is best when a task should execute in isolation, especially for exploration, branching, or large investigations. Small continuity-dependent tasks usually benefit from shared context. The key criterion is preventing unwanted state transfer.

Q4.

A slash command analyzes a bug report and sometimes forms incorrect early hypotheses. The team wants those hypotheses contained so the main session is not biased afterward. What is the best configuration?

A. Use `context: fork` to isolate the analysis B. Increase temperature so later answers are more diverse C. Add more tool permissions D. Disable all memory features globally

Answer: A **Explanation:** Isolating the bug-analysis command prevents speculative hypotheses and dead ends from polluting the parent context. Temperature and permissions do not address state isolation. Global memory changes are much broader than necessary for this problem.

Q5.

Why is `context: fork` valuable for commands that load very large amounts of reference material?

A. It guarantees smaller answers B. It prevents the main session context from being crowded by task-specific material that is irrelevant afterward C. It permanently deletes the loaded material D. It automatically expands tool permissions

Answer: B **Explanation:** Large amounts of task-specific material can consume attention and context budget if kept in the parent session. Forking isolates that load to the subagent, preserving the main session for subsequent work. It does not delete content globally or modify permissions automatically.

Q6.

A team has a slash command `/threat-model` that should independently inspect architecture diagrams and produce a security summary, but the main conversation often shifts back to delivery planning immediately afterward. Which setup is best?

A. Keep `/threat-model` in shared context so delivery planning inherits every security hypothesis B. Use `context: fork` so the threat-modeling session is isolated from later planning C. Replace the command with a static markdown document D. Force the user to start a brand-new session manually every time

Answer: B **Explanation:** Threat modeling often involves exploratory reasoning and large context loads that should not automatically persist into unrelated planning. `context: fork` preserves separation without requiring a totally new user session. Shared context would risk cross-contamination.

Q7.

When should you avoid `context: fork` for a Skill?

A. When the Skill must deliberately build on nuanced details already established in the current session B. When the Skill reads many files C. When the Skill may generate a long answer D. When the Skill uses project standards

Answer: A **Explanation:** If a Skill depends heavily on prior session-specific nuance, isolating it may deprive it of necessary context. Forking is beneficial for separation, but not when continuity is the primary requirement. File reads or long outputs alone do not automatically argue against forking.

Q8.

A configuration author says, "I want this command to act like a side investigation." Which frontmatter option best matches that intent?

A. `context: share` B. `context: fork` C. `output: terse` D. `tools: all`

Answer: B **Explanation:** A side investigation is the canonical mental model for a forked context. The subagent can explore independently and return results without bringing all transient state back into the parent session. Output terseness and tool scope are separate concerns.

Q9.

What problem does `context: fork` primarily solve in Skill and slash command design?

A. Network latency B. Cross-contamination of session state between the main conversation and specialized sub-tasks C. File permission errors D. Lack of markdown support

Answer: B **Explanation:** The main purpose of `context: fork` is isolating session state so specialized work does not pollute the parent thread. It is a context-management mechanism, not a networking or filesystem feature. This is especially important for exploratory or heavy workflows.

Q10.

A command reviews a proposed migration strategy by exploring multiple alternatives, some of which will be discarded. Why is `context: fork` helpful?

A. It keeps discarded alternatives and speculative reasoning out of the parent session's active state B. It increases disk space for the repository C. It forces the command to use fewer tools D. It guarantees the first alternative is correct

Answer: A **Explanation:** Alternative exploration often generates branches of reasoning that are useful locally but noisy afterward. Forking confines those branches to the subagent while still allowing the final conclusion to be returned. It does not guarantee correctness or change storage capacity.

Q11.

A team uses a Skill to summarize vendor documentation. The summary is useful, but the raw document details should not dominate later coding assistance. Which choice is best?

A. Apply `context: fork` so document-heavy processing stays isolated B. Put the full document into the main session every time C. Remove the summary and keep only the raw notes D. Disable slash

commands altogether

Answer: A **Explanation:** Document-heavy work is a common reason to fork context because the raw source material is often not needed after the summary is produced. Isolation keeps later coding help focused. Keeping everything in the main session would waste context capacity and attention.

Q12.

Which statement best distinguishes a forked context from simply writing a longer prompt in the current session?

A. A longer prompt changes wording, while a forked context changes execution isolation and state boundaries B. They are equivalent in all cases C. A longer prompt automatically prevents contamination D. Forking only affects output formatting

Answer: A **Explanation:** Prompt length and context isolation are different mechanisms. `context: fork` creates a separate subagent context boundary, whereas a longer prompt still operates within the existing session state. Only the former directly addresses contamination.

Q13.

A slash command is used for one-off compliance checks that should not affect future brainstorming. What is the best frontmatter decision?

A. Use `context: fork` because the command is specialized and should remain isolated B. Keep shared context because all commands should influence future brainstorming C. Add network access instead D. Remove the command from the project

Answer: A **Explanation:** One-off compliance checks are exactly the kind of specialized task that benefits from isolation. The main session can receive the result without inheriting all the detailed investigative state. Shared context would unnecessarily blend separate modes of work.

Q14.

An author is deciding whether to fork context for a `/rewrite-last-answer` command that depends entirely on the current session's wording and nuance. What is the best choice?

A. Use `context: fork` because all slash commands should be isolated B. Avoid `context: fork` because the command intentionally depends on current-session state C. Fork only if the answer is longer than 500 words D. Add more tool permissions to compensate

Answer: B **Explanation:** A command that rewrites the last answer relies directly on the live session context, so isolation would work against its purpose. `context: fork` is not a universal default; it should be applied where separation is beneficial. Length and permissions are not the deciding factors here.

Q15.

What is the clearest architectural benefit of using `context: fork` for appropriate Skills and slash commands?

A. It allows specialized workflows to run independently while preserving cleanliness and focus in the parent session B. It eliminates the need for prompts C. It grants permanent access to all tools D. It makes every command faster regardless of task

Answer: A **Explanation:** The architectural value is separation of concerns: specialized workflows can operate in their own isolated context and then report back succinctly. This helps maintain a clean parent session and reduces unintended influence from side investigations. Forking does not replace prompting or universally improve speed.
